

Course Code	Course Title	L	R	P	Cr
	AI-Assisted Problem Solving Using Python Programming				
Program Core					

Day 6 Session: Python Libraries & Collections

Day 6

- Lists
- List Comprehensions
- Tuple
- Sets
- Dictionaries

Lists, List Comprehensions, Tuples, Sets & Dictionaries

Topics

- Lists
- List Comprehensions
- Tuples
- Sets
- Dictionaries

Curiosity Question

Think Before We Start!

"How does Amazon store millions of products, WhatsApp store thousands of contacts, and Google Maps manage millions of locations efficiently?"

Discussion

Single variable?

Thousands of variables?

Or special data structures?

Answer: Python Collections

Python Collections:

Collection	Ordered	Mutable	Duplicate Allowed
List	Yes	Yes	Yes
Tuple	Yes	No	Yes
Set	No	Yes	No
Dictionary	Yes	Yes	Keys: No, Values: Yes

Introduction to List

- Lists are used to store multiple items in a single variable.
- Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are [Tuple](#), [Set](#), and [Dictionary](#), all with different qualities and usage.
- The data type list is an ordered sequence which is mutable and made up of one or more elements.
- Unlike a string which consists of only characters, a list can have elements of different data types such as integer, float, string, tuple or even another list.
- A list is very useful to group elements of mixed data types.
- Elements of a list are enclosed in square brackets and are separated by comma.
- Lists are mutable. It means that the contents of the list can be changed after it has been created.

Example:

```
mylist = ["apple", "banana", "cherry"]
```

Example1:

```
#list1 is the list of six even numbers
>>> list1 = [2,4,6,8,10,12]
>>> print(list1)
[2, 4, 6, 8, 10, 12]
```

Example2:

```
#list2 is the list of vowels
>>> list2 = ['a','e','i','o','u']
>>> print(list2)
['a', 'e', 'i', 'o', 'u']
```

- **Example3:**

```
#list3 is the list of mixed data types
>>> list3 = [100,23.5,'Hello']
>>> print(list3)
[100, 23.5, 'Hello']
```

Example4:

```
#list4 is the list of lists called nested
#list
>>> list4 = [['Physics',101],['Chemistry',202],
             ['Mathematics',303]]
>>> print(list4)
[['Physics', 101], ['Chemistry', 202],
 ['Mathematics', 303]]
```

Accessing Elements in a List

- Each element in list is accessed using value called index.
- The first index value is 0, the second index is 1 and so on.
- Elements in the list are assigned index values in increasing order starting from 0.
- To access an element, use square brackets with the index [] value of that element.
- We may also use negative index value to access elements starting from the last element in the list, having index value -1.

Example:

```
#initializing a list named list1
```

```
>>> list1 = [2,4,6,8,10,12]
```

```
>>> list1[0] #returns first element of list1
```

Output: 2

```
>>> list1[3] #returns fourth element of list1
```

Output: 8

```
#Out of range index value for the list returns error
```

```
>>> list1[15]
```

Output: IndexError: list index out of range

```
#an expression resulting in an integer index
```

```
>>> list1[1+4]
```

Output: 12

	0	1	2	3	4	5		Index	
	2	4	6	8	10	12			
	-6	-5	-4	-3	-2	-1		Negative Index	

Lists are Mutable

In Python, lists are mutable.

It means that the contents of the list can be changed after it has been created.

Example:

```
#List list1 of colors
>>> list1 = ['Red', 'Green', 'Blue', 'Orange']
#change/override the fourth element of list1
>>> list1[3] = 'Black'
>>> list1 #print the modified list list1
['Red', 'Green', 'Blue', 'Black']
```

Change a Range of Item Values

To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist[1:4] = ["blackcurrant", "watermelon", "guva"]
print(thislist)
```

```
['apple', 'blackcurrant', 'watermelon', 'guva', 'kiwi', 'mango']
```


List Operations

The data type list allows manipulation of its contents through various operations

1. Concatenation

- Python allows us to join two or more lists using concatenation operator using symbol +.

```
#list1 is list of first five odd integers
>>> list1 = [1,3,5,7,9]

#list2 is list of first five even integers
>>> list2 = [2,4,6,8,10]

#Get elements of list1 followed by list2
>>> list1 + list2
[1, 3, 5, 7, 9, 2, 4, 6, 8, 10]

>>> list3 = ['Red','Green','Blue']
>>> list4 = ['Cyan', 'Magenta', 'Yellow','Black']
>>> list3 + list4
['Red','Green','Blue','Cyan','Magenta','Yellow','Black']
```

- If we try to concatenate a list with elements of some other data type, TypeError occurs.

```
>>> list1 = [1,2,3]
>>> str1 = "abc"
>>> list1 + str1
TypeError: can only concatenate list (not
"str") to list
```

List Operations

2. Repetition

Python allows us to replicate the contents of a list using repetition operator depicted by symbol *.

```
>>> list1 = ['Hello']  
#elements of list1 repeated 4 times  
>>> list1 * 4  
['Hello', 'Hello', 'Hello', 'Hello']
```

3. Membership

The membership operator in checks if the element is present in the list and returns True, else returns False.

```
>>> list1 = ['Red','Green','Blue']  
>>> 'Green' in list1  
True  
>>> 'Cyan' in list1  
False
```

The Operator not in transpose returns True if the element is not present in the list, else it returns False.

```
>>> list1 = ['Red','Green','Blue']  
>>> 'Cyan' not in list1  
True  
>>> 'Green' not in list1  
False
```

List Operations

4. Slicing: Slicing operations allow us to create new list by taking out elements from an existing list.

Syntax: `variable_name[start,end,step]`

```
>>> list1 = ['Red', 'Green', 'Blue', 'Cyan',
'Magenta', 'Yellow', 'Black']
#subject from indexes 2 to 5 of list 1
>>> list1[2:6]
['Blue', 'Cyan', 'Magenta', 'Yellow']

#list1 is truncated to the end of the list
>>> list1[2:20] #second index is out of range
['Blue', 'Cyan', 'Magenta', 'Yellow',
'Black']

>>> list1[7:2]      #first index > second index
[]                  #results in an empty list
```

```
#return sublist from index 0 to 4
>>> list1[:5]      #first index missing
['Red', 'Green', 'Blue', 'Cyan', 'Magenta']

#slicing with a given step size
>>> list1[0:6:2]
['Red', 'Blue', 'Magenta']
#negative indexes
#elements at index -6, -5, -4, -3 are sliced
>>> list1[-6:-2]
['Green', 'Blue', 'Cyan', 'Magenta']

#both first and last index missing
>>> list1[::2]     #step size 2 on entire list
['Red', 'Blue', 'Magenta', 'Black']

#Access list in the reverse order using
negative step size
>>> list1[::-1]
['Black', 'Yellow', 'Magenta', 'Cyan', 'Blue',
'Green', 'Red']
```

Python List Built-in Functions

The data type list has several built-in methods that are useful in programming

Function	Description	Example	Output
len(list)	Returns the number of elements	len([10,20,30])	3
list()	Creates a list	list("ABC")	['A','B','C']
max(list)	Returns the largest element	max([10,30,20])	30
min(list)	Returns the smallest element	min([10,30,20])	10
sum(list)	Returns the sum of elements	sum([10,20,30])	60
sorted(list)	Returns a sorted list	sorted([30,10,20])	[10,20,30]
reversed(list)	Returns reverse iterator	list(reversed([1,2,3]))	[3,2,1]
any(list)	Returns True if at least one element is True	any([0,1,0])	True
all(list)	Returns True if all elements are True	all([1,2,3])	True
enumerate(list)	Returns index and value pairs	list(enumerate(['A','B']))	[(0,'A'),(1,'B')]

Python List Methods

These methods belong to the **list object** and are called using `list_name.method()`.

Method	Description	Example
<code>append(x)</code>	Adds an element at the end	<code>L.append(50)</code>
<code>extend(iterable)</code>	Adds multiple elements	<code>L.extend([60,70])</code>
<code>insert(index, value)</code>	Inserts element at specified position	<code>L.insert(1,15)</code>
<code>remove(value)</code>	Removes first occurrence of value	<code>L.remove(20)</code>
<code>pop()</code>	Removes last element	<code>L.pop()</code>
<code>pop(index)</code>	Removes element at given index	<code>L.pop(2)</code>
<code>clear()</code>	Removes all elements	<code>L.clear()</code>
<code>index(value)</code>	Returns index of element	<code>L.index(30)</code>
<code>count(value)</code>	Counts occurrences	<code>L.count(20)</code>
<code>sort()</code>	Sorts the list	<code>L.sort()</code>
<code>sort(reverse=True)</code>	Sorts in descending order	<code>L.sort(reverse=True)</code>
<code>reverse()</code>	Reverses the list	<code>L.reverse()</code>
<code>copy()</code>	Returns a shallow copy	<code>L.copy()</code>

Traversing a List:

We can access each element of the list or traverse a list using a for loop or a while loop.

List traversal using ***for loop***:

```
>>> list1 = ['Red','Green','Blue','Yellow', 'Black']
>>> for item in list1:
    print(item)
```

Output:

```
Red
Green
Blue
Yellow
Black
```

Another way of accessing the elements of the list is using range() and len() functions:

```
>>> for i in range(len(list1)):
    print(list1[i])
```

Output:

```
Red
Green
Blue
Yellow
Black
```

List Comprehension

Traditional Method

```
squares=[]
```

```
for i in range(1,6):  
    squares.append(i*i)
```

List Comprehension

```
squares=[i*i for i in range(1,6)]
```

```
print(squares)
```

Output

```
[1,4,9,16,25]
```

List Comprehension Examples

```
#Even Numbers
```

```
even=[x for x in range(20) if x%2==0]  
print(even)
```

```
#Uppercase
```

```
names=["ram", "sam", "john"]  
upper=[name.upper() for name in names]  
print(upper)
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]  
['RAM', 'SAM', 'JOHN']
```


Tuple

What is Tuple?

- Tuples are used to store multiple items in a single variable.
- Tuple is one of 4 built-in data types in Python used to store collections of data, the other 3 are [List](#), [Set](#), and [Dictionary](#), all with different qualities and usage.
- A tuple is a collection which is ordered and **unchangeable**.
- Tuples are written with round brackets.

Example:

```
mytuple = ("apple", "banana", "cherry")
```

Tuple Items

Tuple items are ordered, unchangeable, and allow duplicate values.

Tuple items are indexed, the first item has index [0], the second item has index [1] etc...

Tuple Methods

Only Two Methods

Method	Description
<code>count()</code>	Returns the number of times a specified value occurs in a tuple
<code>index()</code>	Searches the tuple for a specified value and returns the position of where it was found

Example

```
numbers=(10,20,10,40)
```

```
print(numbers.count(10))
```

```
print(numbers.index(40))
```

Output

2

3

```
vowels = ('a', 'e', 'i', 'o', 'i', 'u') # vowels tuple
count = vowels.count('i') # count element 'i'
print('The count of i is:', count) # print count
count = vowels.count('p') # count element 'p'
print('The count of p is:', count) # print count
```

The count of i is: 2
The count of p is: 0

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.count(5)
print(x)
```

2

Python Tuple Built-in Functions

Python tuples support several **built-in functions** because they are iterable objects. These functions perform operations on tuples without modifying them (tuples are immutable).

Function	Description	Example	Output
<code>len(tuple)</code>	Returns the number of elements	<code>len((10,20,30))</code>	3
<code>tuple()</code>	Creates a tuple	<code>tuple([1,2,3])</code>	(1,2,3)
<code>max(tuple)</code>	Returns the largest element	<code>max((10,50,30))</code>	50
<code>min(tuple)</code>	Returns the smallest element	<code>min((10,50,30))</code>	10
<code>sum(tuple)</code>	Returns the sum of numeric elements	<code>sum((10,20,30))</code>	60
<code>sorted(tuple)</code>	Returns a sorted list	<code>sorted((30,10,20))</code>	[10,20,30]
<code>reversed(tuple)</code>	Returns a reverse iterator	<code>tuple(reversed((1,2,3)))</code>	(3,2,1)
<code>any(tuple)</code>	Returns True if at least one element is True	<code>any((0,0,1))</code>	True
<code>all(tuple)</code>	Returns True if all elements are True	<code>all((1,2,3))</code>	True
<code>enumerate(tuple)</code>	Returns index-value pairs	<code>list(enumerate(('A','B')))</code>	[(0,'A'),(1,'B')]

Python - Unpack Tuples

Unpacking a Tuple

When we create a tuple, we normally assign values to it. This is called "packing" a tuple:

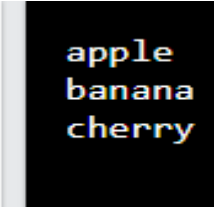
Example: Packing a tuple

```
fruits = ("apple", "banana", "cherry")  
print(fruits)
```

But, in Python, we are also allowed to extract the values back into variables. This is called "unpacking".

Example: Unpacking a tuple

```
fruits = ("apple", "banana", "cherry")  
(green, yellow, red) = fruits  
print(green)  
print(yellow)  
print(red)
```



apple
banana
cherry

Using Asterisk*

If the number of variables is less than the number of values, you can add an * to the variable name and the values will be assigned to the variable as a list

Example

Assign the rest of the values as a list called "red"

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")

(green, yellow, *red) = fruits

print(green)
print(yellow)
print(red)
```

```
apple
banana
['cherry', 'strawberry', 'raspberry']
```

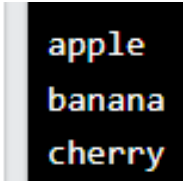
Access Set Items

You cannot access items in a set by referring to an index or a key.

But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in keyword.

Example: Loop through the set, and print the values

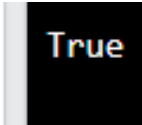
```
thisset = {"apple", "banana", "cherry"}  
for x in thisset:  
    print(x)
```



```
apple  
banana  
cherry
```

Example : Check if "banana" is present in the set

```
thisset = {"apple", "banana", "cherry"}  
print("banana" in thisset)
```



```
True
```

Change Items

Once a set is created, you cannot change its items, but you can add new items.

Python Set Built-in Functions

Sets are unordered collections of **unique elements**. Python provides several built-in functions that work with sets.

Function	Description	Example	Output
<code>len(set)</code>	Returns the number of elements	<code>len({10,20,30})</code>	3
<code>set()</code>	Creates an empty set or converts an iterable into a set	<code>set([1,2,2,3])</code>	{1,2,3}
<code>max(set)</code>	Returns the largest element	<code>max({10,50,30})</code>	50
<code>min(set)</code>	Returns the smallest element	<code>min({10,50,30})</code>	10
<code>sum(set)</code>	Returns the sum of numeric elements	<code>sum({10,20,30})</code>	60
<code>sorted(set)</code>	Returns a sorted list	<code>sorted({30,10,20})</code>	[10,20,30]
<code>any(set)</code>	Returns True if at least one element is True	<code>any({0,0,1})</code>	True
<code>all(set)</code>	Returns True if all elements are True	<code>all({1,2,3})</code>	True
<code>enumerate(set)</code>	Returns index-value pairs	<code>list(enumerate({'A','B'}))</code>	[(0,'A'), (1,'B')] (<i>order may vary</i>)

Method	Description	Example
<code>add(x)</code>	Adds one element	<code>s.add(50)</code>
<code>update(iterable)</code>	Adds multiple elements	<code>s.update([60,70])</code>
<code>remove(x)</code>	Removes an element (error if not found)	<code>s.remove(20)</code>
<code>discard(x)</code>	Removes an element (no error if not found)	<code>s.discard(20)</code>
<code>pop()</code>	Removes and returns a random element	<code>s.pop()</code>
<code>clear()</code>	Removes all elements	<code>s.clear()</code>
<code>copy()</code>	Returns a copy of the set	<code>s.copy()</code>
<code>union()</code>	Returns the union of sets	<code>A.union(B)</code>
<code>intersection()</code>	Returns common elements	<code>A.intersection(B)</code>
<code>difference()</code>	Returns elements in first set only	<code>A.difference(B)</code>
<code>symmetric_difference()</code>	Returns elements in either set but not both	<code>A.symmetric_difference(B)</code>
<code>issubset()</code>	Checks if one set is a subset of another	<code>A.issubset(B)</code>
<code>issuperset()</code>	Checks if one set is a superset of another	<code>A.issuperset(B)</code>
<code>isdisjoint()</code>	Checks if two sets have no common elements	<code>A.isdisjoint(B)</code>
<code>intersection_update()</code>	Updates set with common elements	<code>A.intersection_update(B)</code>
<code>difference_update()</code>	Removes common elements from the first set	<code>A.difference_update(B)</code>
<code>symmetric_difference_update()</code>	Updates set with symmetric difference	<code>A.symmetric_difference_update(B)</code>

Dictionary

- The data type dictionary falls under mapping.
- It is a mapping between a set of keys and a set of values.
- The key-value pair is called an item.
- A key is separated from its value by a colon(:) and consecutive items are separated by commas.
- Items in dictionaries are unordered, so we may not get back the data in the same order in which we had entered the data initially in the dictionary.
- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

Example:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

Python Dictionary Built-in Functions

A **dictionary** stores data as **key-value pairs**. Python provides several built-in functions that work with dictionaries.

Function	Description	Example	Output
<code>len(dict)</code>	Returns the number of key-value pairs	<code>len({'A':10,'B':20})</code>	2
<code>dict()</code>	Creates a dictionary	<code>dict(name='John', age=20)</code>	<code>{'name':'John','age':20}</code>
<code>max(dict)</code>	Returns the largest key	<code>max({'A':10,'C':30,'B':20})</code>	'C'
<code>min(dict)</code>	Returns the smallest key	<code>min({'A':10,'C':30,'B':20})</code>	'A'
<code>sum(dict.values())</code>	Returns the sum of numeric values	<code>sum({'A':10,'B':20}.values())</code>	30
<code>sorted(dict)</code>	Returns a sorted list of keys	<code>sorted({'B':20,'A':10})</code>	['A','B']
<code>any(dict)</code>	Returns True if at least one key is True	<code>any({1:'A',0:'B'})</code>	True
<code>all(dict)</code>	Returns True if all keys are True	<code>all({1:'A',2:'B'})</code>	True
<code>list(dict)</code>	Returns a list of keys	<code>list({'A':10,'B':20})</code>	['A','B']

Method	Description	Example
<code>clear()</code>	Removes all items	<code>d.clear()</code>
<code>copy()</code>	Returns a shallow copy	<code>d.copy()</code>
<code>fromkeys()</code>	Creates a dictionary from keys	<code>dict.fromkeys(keys,0)</code>
<code>get()</code>	Returns the value of a key	<code>d.get('name')</code>
<code>items()</code>	Returns key-value pairs	<code>d.items()</code>
<code>keys()</code>	Returns all keys	<code>d.keys()</code>
<code>values()</code>	Returns all values	<code>d.values()</code>
<code>pop()</code>	Removes specified key	<code>d.pop('age')</code>
<code>popitem()</code>	Removes the last inserted item	<code>d.popitem()</code>
<code>setdefault()</code>	Returns value if key exists, otherwise inserts key	<code>d.setdefault('city','Hyderabad')</code>
<code>update()</code>	Updates dictionary with another dictionary	<code>d.update({'age':22})</code>

Dictionary Comprehension

Dictionary comprehension is used to create a dictionary in a short and clear way. It allows keys and values to be generated from a loop in one line.

This helps in building dictionaries directly without writing multiple statements.

Example:

This example creates a dictionary where numbers from 1 to 5 are used as keys and their squares are stored as values.

Examples:

```
sq = {x: x**2 for x in range(1, 6)}  
print(sq)
```

Output{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Even or Odd

```
result = {x: "Even" if x % 2 == 0 else "Odd" for x in range(1, 11)}  
print(result)
```

Only Even Numbers (Filter Condition)

```
result = {x: "Even" for x in range(1, 11) if x % 2 == 0}  
print(result)
```



Thank you